

CRD Advanced: Contrasts and Multiple Comparisons

Multiple Comparisons and Post-Hoc Tests

Dr. Samuel B Fernandes

2026-01-01

Learning Objectives

- Understand when to use **planned contrasts** vs. **post-hoc comparisons**
- Apply **orthogonal contrasts** to test specific hypotheses in CRD
- Implement multiple comparison procedures: **Tukey HSD**, **Dunnett's test**
- Use **emmeans** package for estimating marginal means and contrasts

Review: Completely Randomized Design (CRD)

CRD Model:

$$y_{ij} = \mu + \tau_i + \epsilon_{ij} \text{ or } y_{ij} = \mu_i + \epsilon_{ij}$$

Where:

- y_{ij} = observation j in treatment i
- μ = overall mean
- τ_i = effect of treatment i
- $\epsilon_{ij} \sim N(0, \sigma^2)$ = random error

Key assumption: Experimental units are **homogeneous** (no blocking needed)

After Significant F-test: What Now?

Significant F-test means:

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_t \text{ is rejected}$$

At least one mean differs, but which ones?

Two approaches:

1. **Planned comparisons (contrasts):** Pre-specified hypotheses
2. **Post-hoc comparisons:** Explore all pairwise differences

Diagnostic Plots: Check Assumptions First!

Before interpreting ANOVA, always check assumptions:

1. **Normality of residuals:** Q-Q plot
2. **Homogeneity of variance:** Residuals vs. Fitted
3. **Independence:** Random assignment in CRD
4. **No extreme outliers:** Leverage plot

```
# Example: Variety trial
data(archbold.apple, package = "agridat")
apple <- archbold.apple |>
  filter(!is.na(yield)) |>
  mutate(gen = factor(gen))

# Fit CRD model
model_apple <- lm(yield ~ gen, data = apple)

# Create diagnostic plots
par(mfrow = c(2, 2))
plot(model_apple)
par(mfrow = c(1, 1))
```

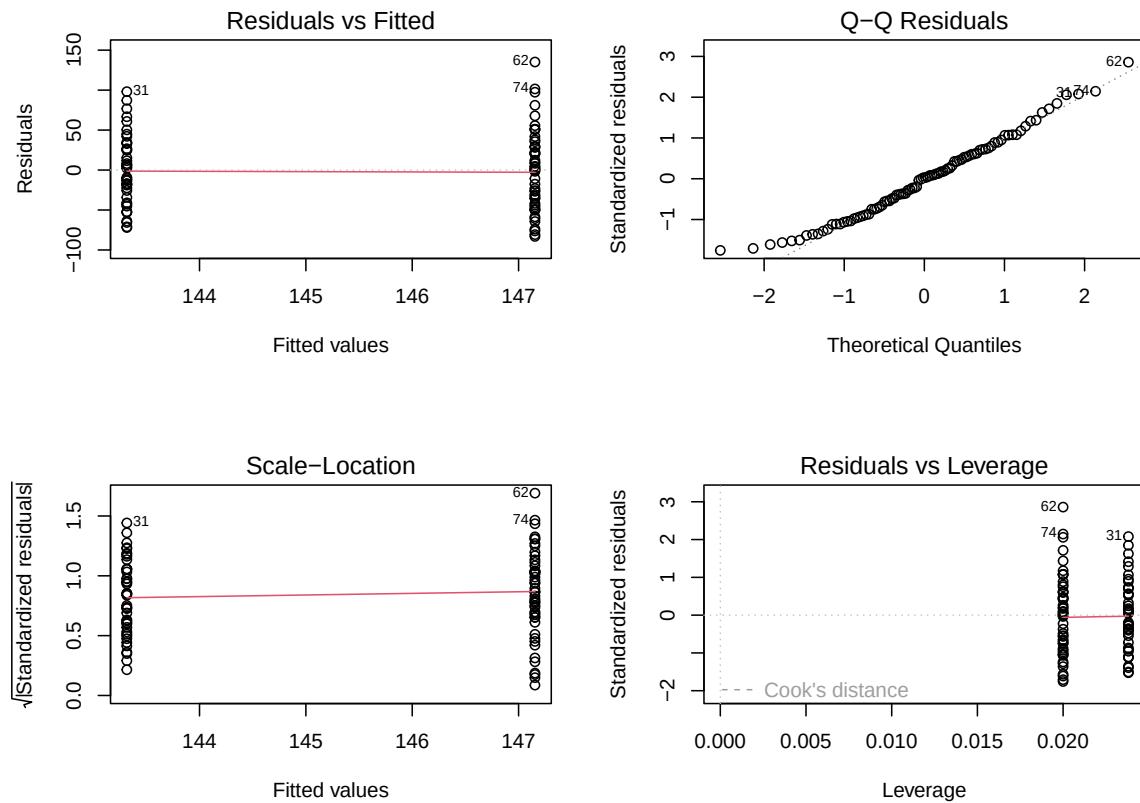


Figure 1: Four diagnostic plots for checking ANOVA assumptions in CRD

Real Data Example: Radon Release Experiment

Laboratory CRD: Effect of orifice diameter on radon release

```
# Load radon data from GitHub
radon_url <- "https://raw.githubusercontent.com/Jaisai0611/CRD-example/main/EX%203.21.csv"
radon_data <- read.csv(radon_url) |>
  pivot_longer(
    cols = starts_with("Radon"),
    names_to = "sample",
    values_to = "radon_pct"
  ) |>
  mutate(
    orifice = factor(Orifice.Diameter),
```

```

      response = radon_pct
    ) |>
    dplyr::select(orifice, sample, response)

# Summary statistics by orifice size
radon_data |>
  group_by(orifice) |>
  summarize(
    n = n(),
    mean = mean(response),
    sd = sd(response),
    .groups = "drop"
  ) |>
  knitr::kable(digits = 1)

```

orifice	n	mean	sd
0.37	4	82.8	2.1
0.51	4	77.0	2.3
0.71	4	75.0	1.8
1.02	4	71.8	3.3
1.4	4	65.0	3.6
1.99	4	62.8	2.8

Visualize Treatment Effects

```

ggplot(radon_data, aes(x = orifice, y = response, fill = orifice)) +
  geom_boxplot(alpha = 0.7) +
  geom_jitter(width = 0.15, alpha = 0.5, size = 2.5) +
  labs(
    title = "Effect of Orifice Diameter on Radon Release",
    x = "Orifice Diameter (mm)",
    y = "Radon Released (%)",
    fill = "Diameter"
  ) +
  scale_fill_viridis_d(option = "plasma") +
  theme_minimal() +
  theme(
    text = element_text(size = 14),

```

```

legend.position = "none"
)

```

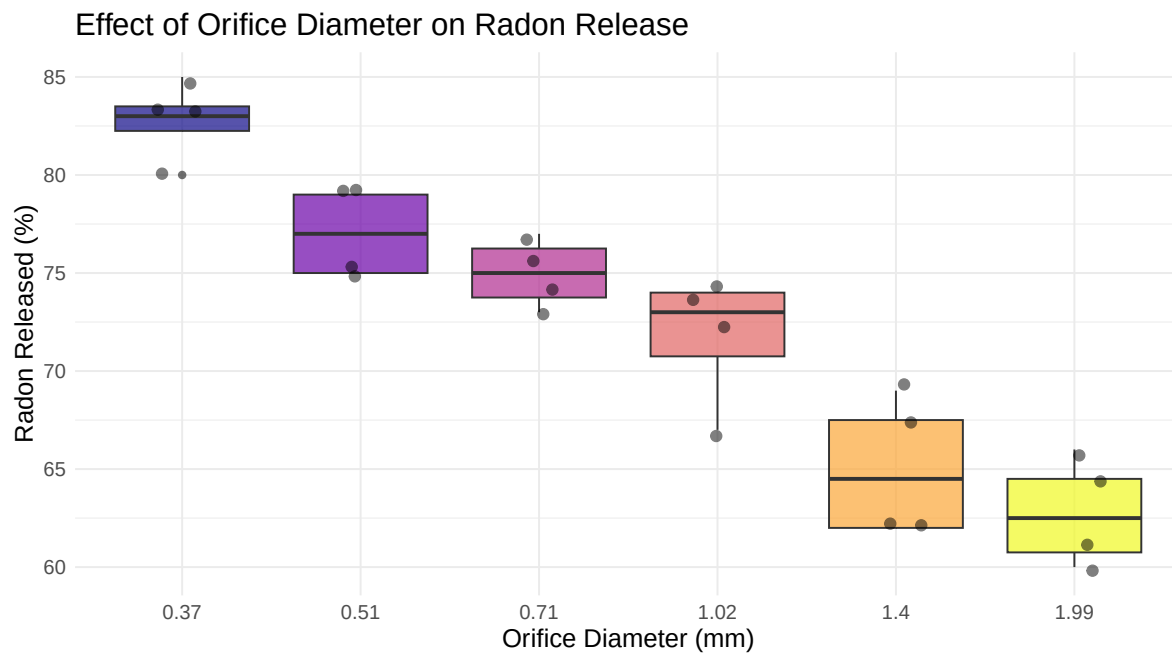


Figure 2: Radon release percentage by orifice diameter

Fit CRD Model and Check Assumptions

Fit one-way ANOVA model

```
model_radon <- lm(response ~ orifice, data = radon_data)
```

ANOVA table

```
anova(model_radon)
```

```

#> Analysis of Variance Table
#>
#> Response: response
#>           Df Sum Sq Mean Sq F value    Pr(>F)
#> orifice     5 1133.38  226.675   30.852 3.16e-08 ***
#> Residuals  18  132.25    7.347

```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Model coefficients

```
summary(model_radon)
```

```
#>
#> Call:
#> lm(formula = response ~ orifice, data = radon_data)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#>  -4.75  -2.00   0.25   2.00   4.00
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   82.750      1.355   61.057 < 2e-16 ***
#> orifice0.51  -5.750      1.917   -3.000 0.007685 **
#> orifice0.71  -7.750      1.917   -4.043 0.000762 ***
#> orifice1.02 -11.000      1.917   -5.739 1.93e-05 ***
#> orifice1.4   -17.750      1.917   -9.261 2.87e-08 ***
#> orifice1.99 -20.000      1.917  -10.435 4.62e-09 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 2.711 on 18 degrees of freedom
#> Multiple R-squared:  0.8955, Adjusted R-squared:  0.8665
#> F-statistic: 30.85 on 5 and 18 DF, p-value: 3.16e-08
```

Diagnostic Plots: Radon CRD

```
# Check ANOVA assumptions
par(mfrow = c(2, 2), mar = c(4, 4, 2, 2))
plot(model_radon)
par(mfrow = c(1, 1))
```

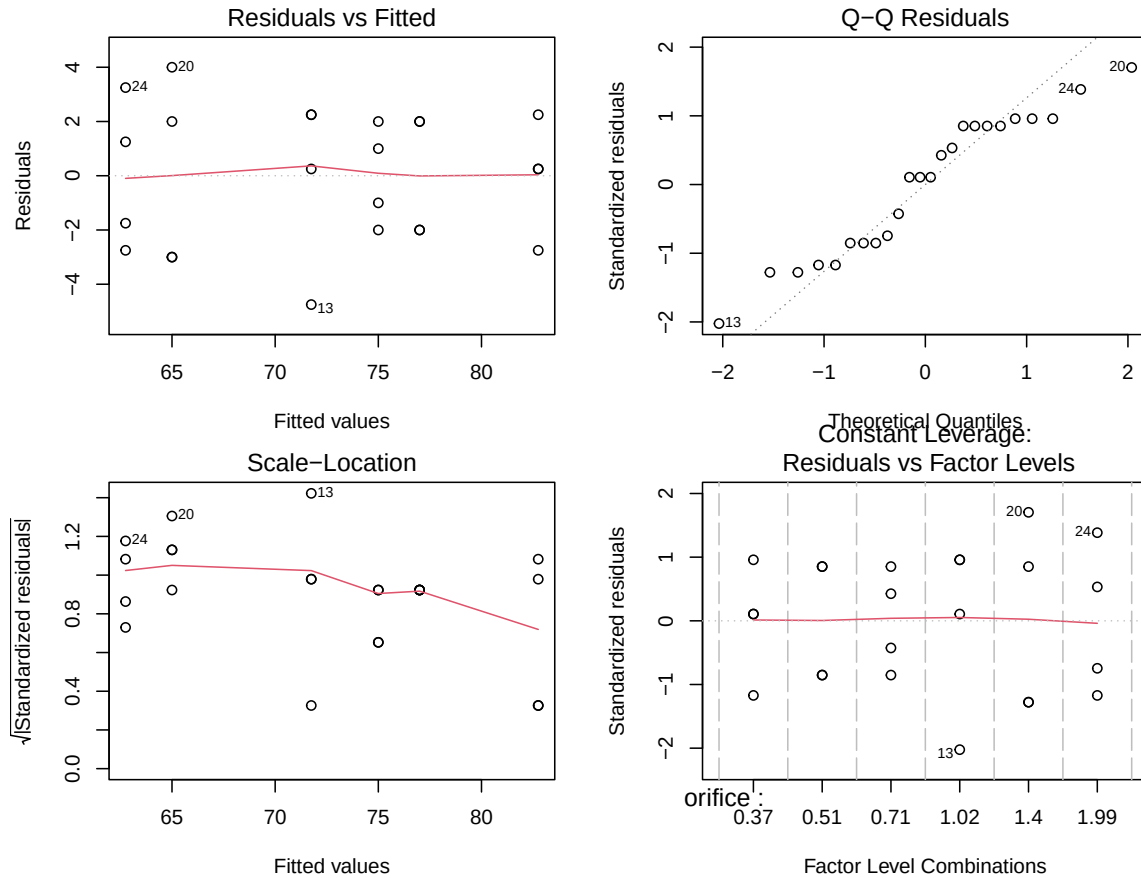


Figure 3: Diagnostic plots for radon release CRD

Estimated Marginal Means with emmeans

Calculate estimated marginal means (EMMs)

```
emm_radon <- emmeans(model_radon, ~orifice)
emm_radon
```

```
#> orifice emmean   SE df lower.CL upper.CL
#> 0.37      82.8 1.36 18    79.9    85.6
#> 0.51      77.0 1.36 18    74.2    79.8
#> 0.71      75.0 1.36 18    72.2    77.8
#> 1.02      71.8 1.36 18    68.9    74.6
#> 1.4       65.0 1.36 18    62.2    67.8
#> 1.99      62.8 1.36 18    59.9    65.6
```

```
#>
#> Confidence level used: 0.95
```

Display with confidence intervals

```
TukeyHSD(aov(response ~ orifice, data = radon_data))
```

```
#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>
#> Fit: aov(formula = response ~ orifice, data = radon_data)
#>
#> $orifice
#>      diff      lwr      upr      p adj
#> 0.51-0.37 -5.75 -11.841234  0.3412336 0.0707511
#> 0.71-0.37 -7.75 -13.841234 -1.6587664 0.0084181
#> 1.02-0.37 -11.00 -17.091234 -4.9087664 0.0002404
#> 1.4-0.37  -17.75 -23.841234 -11.6587664 0.0000004
#> 1.99-0.37 -20.00 -26.091234 -13.9087664 0.0000001
#> 0.71-0.51  -2.00  -8.091234  4.0912336 0.8968057
#> 1.02-0.51  -5.25 -11.341234  0.8412336 0.1153360
#> 1.4-0.51  -12.00 -18.091234 -5.9087664 0.0000841
#> 1.99-0.51 -14.25 -20.341234 -8.1587664 0.0000089
#> 1.02-0.71  -3.25  -9.341234  2.8412336 0.5513482
#> 1.4-0.71  -10.00 -16.091234 -3.9087664 0.0007059
#> 1.99-0.71 -12.25 -18.341234 -6.1587664 0.0000650
#> 1.4-1.02   -6.75 -12.841234 -0.6587664 0.0249971
#> 1.99-1.02  -9.00 -15.091234 -2.9087664 0.0021152
#> 1.99-1.4   -2.25  -8.341234  3.8412336 0.8432736
```

Visualize Estimated Marginal Means

```
# Plot EMMs with CIs
plot(emm_radon, comparisons = TRUE) +
  labs(
    title = "Radon Release by Orifice Diameter with 95% CIs",
    x = "Radon Released (%)",
    y = "Orifice Diameter (mm)"
  ) +
  theme_minimal() +
  theme(text = element_text(size = 13))
```

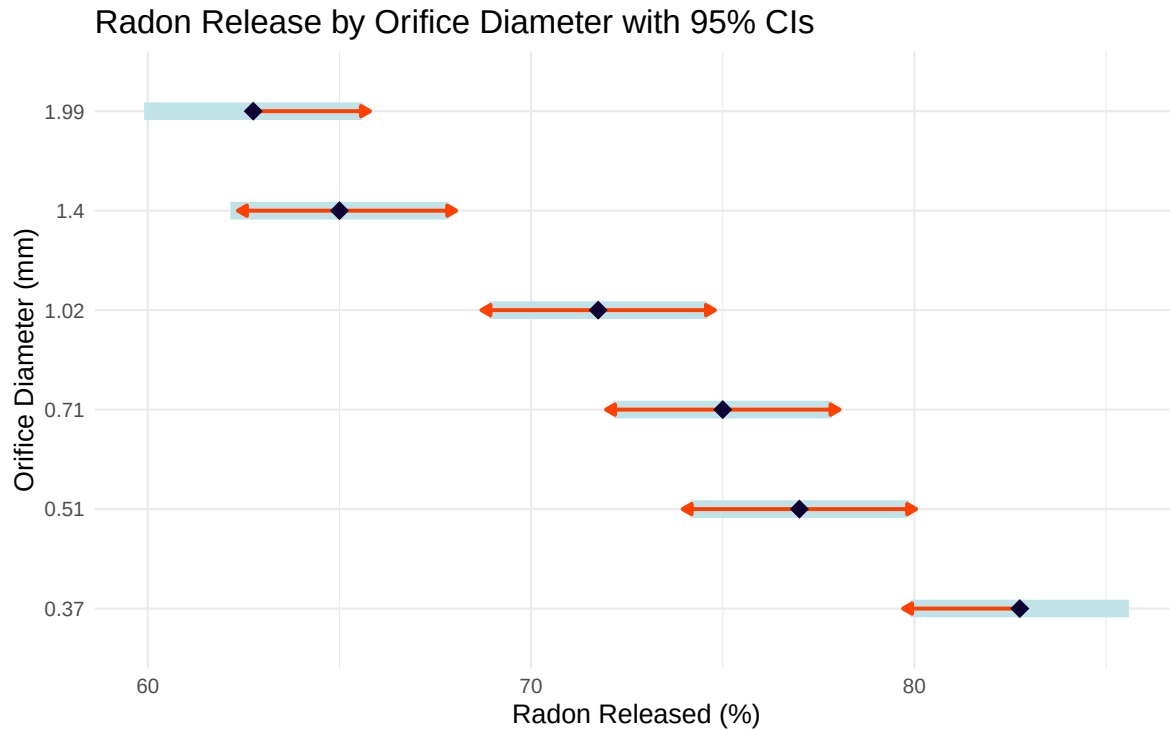


Figure 4: Estimated marginal means with 95% confidence intervals

Planned Contrasts: Testing Specific Hypotheses

Scientific questions can be translated into contrasts.

Example with 4 NPK application methods:

1. No NPK (control)
2. NPK in January (plowed)
3. NPK in January (broadcast)
4. NPK in April (broadcast)

Research questions → Contrasts:

- C_1 : Does NPK help? (Control vs. all NPK treatments)
- C_2 : Does timing matter? (January vs. April application)
- C_3 : Does method matter? (Plowed vs. broadcast within January)

Contrast Coefficients: How to Construct

Treatments:

1. No NPK (Control)
2. Jan (plowed)
3. Jan (broadcast)
4. Apr (broadcast)

Contrasts:

Contrast	1	2	3	4
C_1 : Control vs NPK	+3	-1	-1	-1
C_2 : Jan vs Apr	0	+1	+1	-2
C_3 : Plow vs Broadcast	0	+1	-1	0

Rules for contrast coefficients:

1. Must sum to zero: $\sum c_i = 0$
2. For orthogonality: $\sum c_{1i} \cdot c_{2i} = 0$

Simulated Example: NPK Fertilizer Trial

```
# Create simulated but realistic NPK application data
set.seed(2026)
npk_data <- data.frame(
  treatment = factor(rep(c("Control", "Jan_Plow", "Jan_Broadcast", "Apr_Broadcast"), each = 8),
    levels = c("Control", "Jan_Plow", "Jan_Broadcast", "Apr_Broadcast")
  ),
  block = factor(rep(1:8, 4)),
  yield = c(
    rnorm(8, 45, 5), # Control: lower yield
    rnorm(8, 62, 5), # Jan plowed: best
    rnorm(8, 58, 5), # Jan broadcast: good
    rnorm(8, 55, 6) # Apr broadcast: moderate
  )
)

# Summary by treatment
npk_data |>
```

```
group_by(treatment) |>
summarize(
  n = n(),
  mean = mean(yield),
  sd = sd(yield),
  .groups = "drop"
) |>
knitr::kable(digits = 1)
```

treatment	n	mean	sd
Control	8	41.6	4.7
Jan_Plow	8	60.0	5.4
Jan_Broadcast	8	58.2	4.5
Apr_Broadcast	8	59.3	5.6

Visualize NPK Trial Data

```
ggplot(npk_data, aes(x = treatment, y = yield, fill = treatment)) +
  geom_boxplot(alpha = 0.7) +
  geom_jitter(width = 0.15, alpha = 0.5, size = 2.5) +
  labs(
    title = "Effect of NPK Application Method on Crop Yield",
    x = "Treatment",
    y = "Yield (bushels/acre)",
    fill = "Treatment"
  ) +
  scale_fill_viridis_d(option = "plasma") +
  theme_minimal() +
  theme(
    text = element_text(size = 14),
    axis.text.x = element_text(angle = 30, hjust = 1),
    legend.position = "none"
  )
```

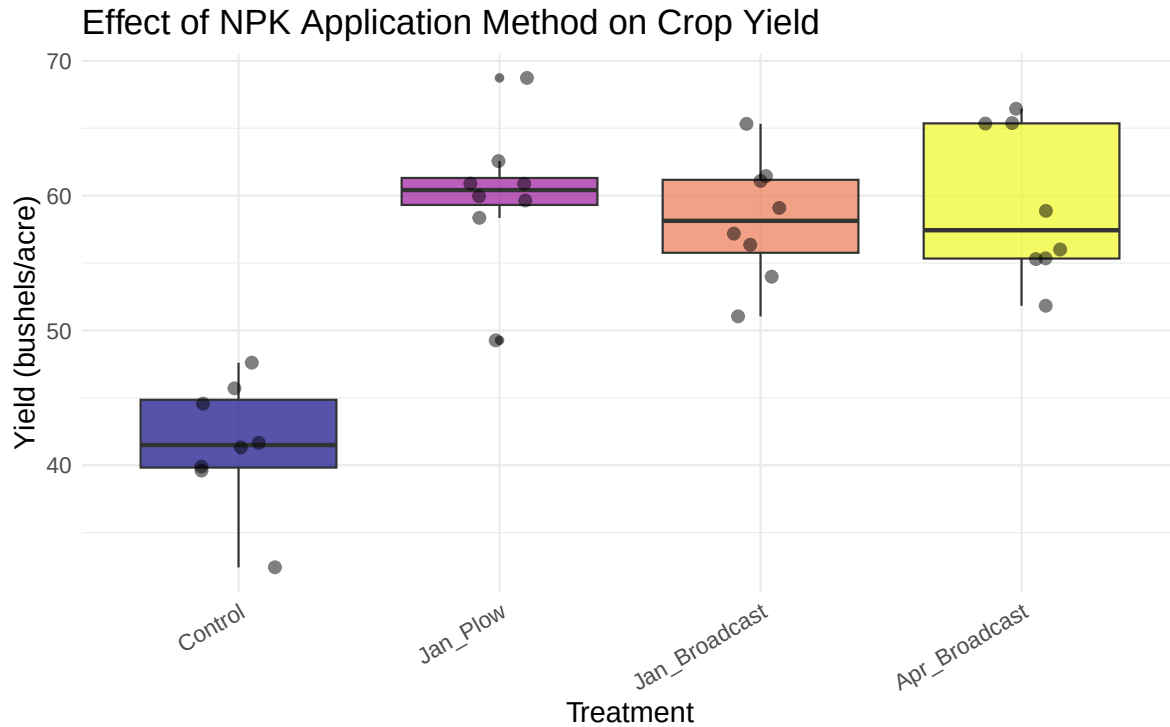


Figure 5: Crop yield by NPK application method

Implementing Contrasts with emmeans

```
# Fit CRD model
model_npk <- lm(yield ~ treatment, data = npk_data)

# Get estimated marginal means
emm_npk <- emmeans(model_npk, ~treatment)

# Define contrasts as a list
contrast_list <- list(
  "Control vs NPK" = c(3, -1, -1, -1) / 3, # Divide to get average
  "Jan vs Apr" = c(0, 1, 1, -2) / 2, # Compare averages
  "Plow vs Broadcast" = c(0, 1, -1, 0) # Simple comparison
)

# Test contrasts
contrast_results <- contrast(emm_npk, contrast_list)
print(contrast_results)
```

```
#> contrast      estimate    SE df t.ratio p.value
#> Control vs NPK    -17.580 2.07 28  -8.475 <.0001
#> Jan vs Apr        -0.205 2.20 28  -0.093 0.9265
#> Plow vs Broadcast   1.847 2.54 28   0.727 0.4734
```

```
# With confidence intervals
confint(contrast_results)
```

```
#> contrast      estimate    SE df lower.CL upper.CL
#> Control vs NPK    -17.580 2.07 28  -21.83  -13.33
#> Jan vs Apr        -0.205 2.20 28   -4.71   4.30
#> Plow vs Broadcast   1.847 2.54 28   -3.36   7.05
#>
#> Confidence level used: 0.95
```

Visualize Contrast Results

```
# Plot contrasts with confidence intervals
contrast_df <- confint(contrast_results) |> as.data.frame()
if (!all(c("lower.CL", "upper.CL") %in% names(contrast_df))) {
  contrast_df <- contrast_df |>
    mutate(
      lower.CL = estimate - 1.96 * SE,
      upper.CL = estimate + 1.96 * SE
    )
}

contrast_df |>
  ggplot(aes(x = contrast, y = estimate)) +
  geom_point(size = 4, color = "steelblue") +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL),
    width = 0.2, linewidth = 1.2, color = "steelblue"
  ) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red", linewidth = 0.8) +
  coord_flip() +
  labs(
    title = "Planned Contrasts: NPK Application Methods",
    x = "Contrast",
```

```

y = "Effect Size (bushels/acre)",
caption = "Error bars show 95% confidence intervals. Red line at zero = no effect."
) +
theme_minimal() +
theme(text = element_text(size = 13))

```

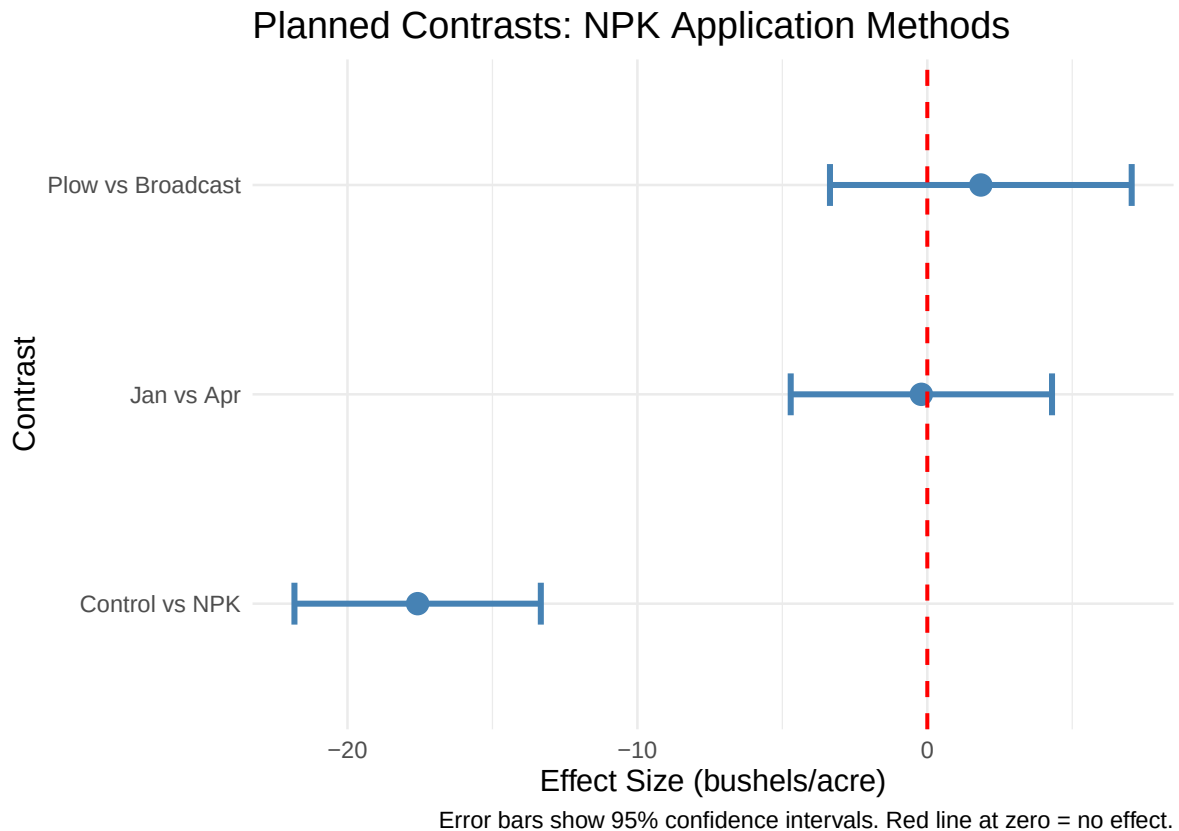


Figure 6: Contrast estimates with 95% confidence intervals

Post-Hoc Comparisons: Multiple Testing Problem

Use post-hoc tests when:

- No pre-planned hypotheses
- Exploratory analysis
- Want to compare **all pairs** of treatments

Problem: Familywise Error Rate

$$\alpha_{familywise} = 1 - (1 - \alpha)^k$$

where k = number of comparisons

Example: 5 treatments $\rightarrow \binom{5}{2} = 10$ pairwise comparisons

- Without adjustment: $\alpha_{FW} = 1 - (0.95)^{10} = 0.40$
- **40% chance of at least one false positive!**

Terminology: Familywise error rate and **experimentwise error rate** are the same thing—the probability of making at least one Type I error across all comparisons in the experiment.

Tukey's Honest Significant Difference (HSD)

Most common post-hoc test for all pairwise comparisons

$$HSD = q_{\alpha, t, df_{error}} \times \sqrt{\frac{MS_{Error}}{n}}$$

- q = studentized range statistic (not t-distribution!)
- t = number of treatments
- Properly controls familywise error rate at α

Properties:

- Conservative (low Type I error)
- Good for balanced or nearly balanced designs
- Compares all treatment pairs

Tukey HSD with emmeans: Compact Letter Display

Pairwise comparisons with Tukey adjustment

```
tukey_results <- pairs(emm_npk, adjust = "tukey")
tukey_results
```

```
#> contrast estimate SE df t.ratio p.value
#> Control - Jan_Plow -18.435 2.54 28 -7.256 <.0001
#> Control - Jan_Broadcast -16.589 2.54 28 -6.530 <.0001
#> Control - Apr_Broadcast -17.717 2.54 28 -6.974 <.0001
#> Jan_Plow - Jan_Broadcast 1.847 2.54 28 0.727 0.8855
```

```
#> Jan_Plow - Apr_Broadcast      0.718 2.54 28    0.283  0.9919
#> Jan_Broadcast - Apr_Broadcast -1.128 2.54 28   -0.444  0.9702
#>
#> P value adjustment: tukey method for comparing a family of 4 estimates
```

More intuitive way to present multiple comparisons:

```
cld_tukey <- cld(emm_npk, adjust = "tukey", Letters = letters)
cld_tukey |>
  as.data.frame() |>
  knitr::kable(digits = 1)
```

	treatment	emmean	SE	df	lower.CL	upper.CL	.group
1	Control	41.6	1.8	28	36.8	46.4	a
3	Jan_Broadcast	58.2	1.8	28	53.4	63.0	b
4	Apr_Broadcast	59.3	1.8	28	54.5	64.1	b
2	Jan_Plow	60.0	1.8	28	55.3	64.8	b

Reading CLDs: Treatments that share a letter are **not significantly different**. Treatments with completely different letters are **significantly different**.

For example:

- Treatments marked “a” and “ab” share the letter “a”, so they’re not significantly different
- Treatments marked “a” and “c” share no letters, so they ARE significantly different

Visualize Tukey Results with Compact Letters

Compact Letter Display Table:

	treatment	emmean	SE	df	lower.CL	upper.CL	.group
1	Control	41.6	1.8	28	36.8	46.4	a
3	Jan_Broadcast	58.2	1.8	28	53.4	63.0	b
4	Apr_Broadcast	59.3	1.8	28	54.5	64.1	b
2	Jan_Plow	60.0	1.8	28	55.3	64.8	b

How to read CLDs:

- Treatments with **same letters** = not significantly different
- Treatments with **no shared letters** = significantly different
- Example: “a” vs “c” = different; “ab” vs “b” = same letter “b” = not different

```
# Combine EMMS with CLD letters for plotting
cld_df <- cld_tukey |>
  as.data.frame() |>
  mutate(treatment = factor(treatment, levels = treatment))

ggplot(cld_df, aes(x = treatment, y = emmean)) +
  geom_point(size = 5, color = "darkgreen") +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL),
    width = 0.3, linewidth = 1.2, color = "darkgreen"
  ) +
  geom_text(aes(label = str_trim(.group)),
    vjust = -1.5, size = 5, fontface = "bold", color = "red"
  ) +
  labs(
    title = "NPK Treatment Effects with Tukey Groupings",
    x = "Treatment",
    y = "Estimated Mean Yield (bushels/acre)"
  ) +
  scale_y_continuous(limits = c(40, 72)) +
  theme_minimal() +
  theme(
    text = element_text(size = 12),
    axis.text.x = element_text(angle = 30, hjust = 1)
  )
```

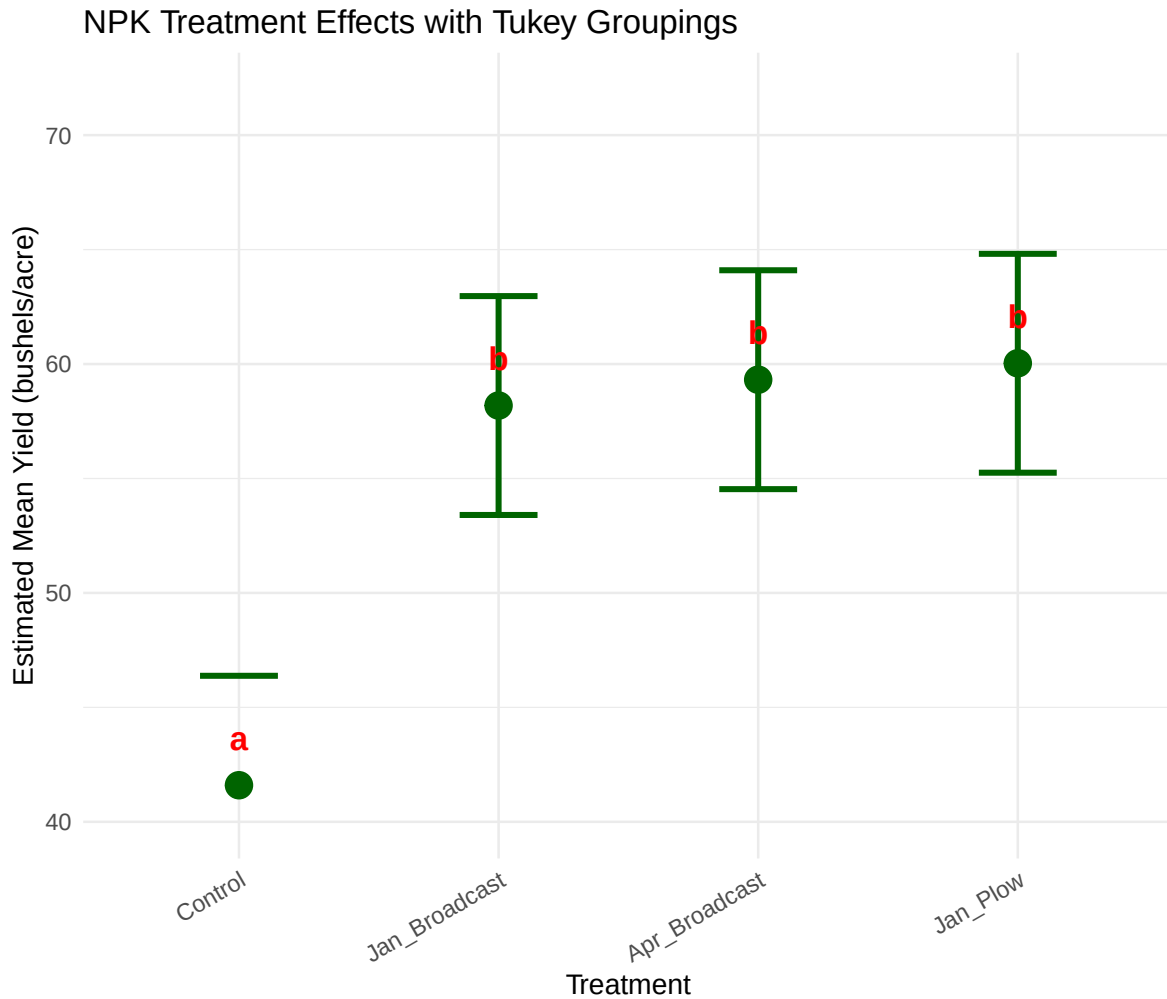


Figure 7: Treatment means with 95% CI and Tukey HSD groups

Dunnett's Test: Compare Treatments to Control

Use when: You have a **control** and want to compare all treatments to it

$$d = t_{Dunnett} \times \sqrt{\frac{2 \times MS_{Error}}{n}}$$

Advantages over Tukey:

- More **powerful** for control vs. treatment comparisons
- Doesn't "waste" comparisons on treatment-treatment pairs

- Still controls familywise error rate

Common in agriculture: Herbicide trials, variety comparisons to standard

Dunnett's Test Implementation

```
# Dunnett's test: all treatments vs Control
# Control must be the reference level (first level of factor)
dunnett_results <- contrast(emm_npk, method = "trt.vs.ctrl", adjust = "dunnett")
dunnett_results |>
  as.data.frame() |>
  knitr::kable(digits = 2)
```

contrast	estimate	SE	df	t.ratio	p.value
Jan_Plow - Control	18.44	2.54	28	7.26	0
Jan_Broadcast - Control	16.59	2.54	28	6.53	0
Apr_Broadcast - Control	17.72	2.54	28	6.97	0

```
# With confidence intervals
confint(dunnett_results) |>
  as.data.frame() |>
  knitr::kable(digits = 2)
```

contrast	estimate	SE	df	lower.CL	upper.CL
Jan_Plow - Control	18.44	2.54	28	12.09	24.78
Jan_Broadcast - Control	16.59	2.54	28	10.24	22.93
Apr_Broadcast - Control	17.72	2.54	28	11.37	24.06

Alternative: rstatix for Tidy Workflows

Modern tidyverse-friendly approach using rstatix:

```
# Complete workflow: emmeans_test() - pipe-friendly!
rstatix_results <- npk_data |>
  emmeans_test(yield ~ treatment, p.adjust.method = "tukey")

# Display key columns
```

```
rstatix_results |>
  select(term, .y., group1, group2, statistic, p, p.adj, p.adj.signif) |>
  knitr::kable(digits = 3)
```

term	.y.	group1	group2	statistic	p	p.adj	p.adj.signif
treat- ment	yield	Control	Jan_Plow	-7.256	0.000	0.000	****
treat- ment	yield	Control	Jan_Broad- cast	-6.530	0.000	0.000	****
treat- ment	yield	Control	Apr_Broad- cast	-6.974	0.000	0.000	****
treat- ment	yield	Jan_Plow	Jan_Broad- cast	0.727	0.473	0.979	ns
treat- ment	yield	Jan_Plow	Apr_Broad- cast	0.283	0.779	1.000	ns
treat- ment	yield	Jan_Broad- cast	Apr_Broad- cast	-0.444	0.660	0.998	ns

Tip

Why rstatix? Integrates seamlessly with dplyr pipes and returns tidy data frames (not emmGrid objects). Perfect for automated reporting and further data manipulation.

Alternative Approach: `rstatix tukey_hsd()`

Even simpler for pairwise comparisons:

```
# rstatix provides a simple tukey_hsd() function
npk_tukey <- npk_data |>
  tukey_hsd(yield ~ treatment)

# Display pairwise comparison results
npk_tukey |>
  select(group1, group2, estimate, conf.low, conf.high, p.adj, p.adj.signif) |>
  knitr::kable(digits = 2)
```

group1	group2	estimate	conf.low	conf.high	p.adj	p.adj.signif
Control	Jan_Plow	18.44	11.50	25.37	0.00	****
Control	Jan_Broad-	16.59	9.65	23.53	0.00	****
	cast					
Control	Apr_Broad-	17.72	10.78	24.65	0.00	****
	cast					
Jan_Plow	Jan_Broad-	-1.85	-8.78	5.09	0.89	ns
	cast					
Jan_Plow	Apr_Broad-	-0.72	-7.65	6.22	0.99	ns
	cast					
Jan_Broad-	Apr_Broad-	1.13	-5.81	8.06	0.97	ns
cast	cast					

What rstatix provides: Pairwise comparisons with p-values and significance stars (*, **, ***).

Key advantages:

- Pipe-friendly syntax fits tidyverse workflows
- Output is already a data frame (no conversion needed)
- Great for students who prefer dplyr-style code

Important: For CLDs, use `emmeans::cld()` as shown in the previous slide.

ggpubr: Publication-Ready Comparison Plots

Complete tidyverse + ggpubr workflow:

```
# Step 1: Run statistical test using rstatix
pwc <- npk_data |>
  emmeans_test(yield ~ treatment, p.adjust.method = "tukey")

# Step 2: Prepare positions for brackets
pwc <- pwc |> add_xy_position(x = "treatment")

# Step 3: Create plot with automated significance brackets
ggplot(npk_data, aes(x = treatment, y = yield, fill = treatment)) +
  geom_boxplot(alpha = 0.7, outlier.shape = NA) +
  geom_jitter(width = 0.15, alpha = 0.5, size = 2) +
  # Step 4: Add significance brackets automatically
  stat_pvalue_manual(pwc,
```

```

hide.ns = TRUE, label = "p.adj.signif",
tip.length = 0.01, step.increase = 0.05
) +
labs(
  title = "NPK Treatment Effects with Automated Significance Testing",
  subtitle = "Brackets show significant pairwise differences (Tukey HSD,  $\alpha = 0.05$ )",
  x = "Treatment",
  y = "Yield (bushels/acre)",
  fill = "Treatment"
) +
scale_fill_viridis_d(option = "plasma") +
theme_minimal() +
theme(
  text = element_text(size = 13),
  axis.text.x = element_text(angle = 30, hjust = 1),
  legend.position = "none"
)

```

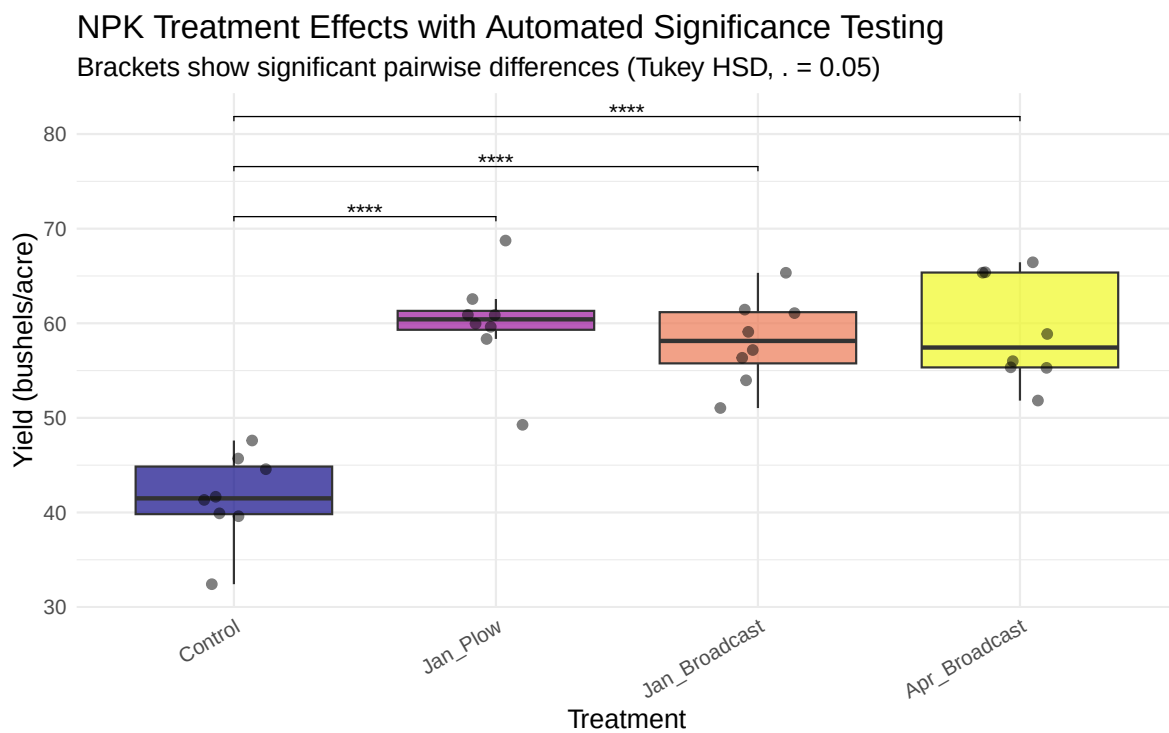


Figure 8: NPK treatment comparison with ggpubr automated annotations

i Note

ggpubr advantage: Automatically adds significance brackets to plots—perfect for presentations and publications. Uses rstatix output directly!

Comparison of Adjustment Methods

```
# Compare different adjustment methods
methods_comparison <- list(
  Tukey = as.data.frame(pairs(emm_npk, adjust = "tukey")),
  Bonferroni = as.data.frame(pairs(emm_npk, adjust = "bonferroni")),
  None = as.data.frame(pairs(emm_npk, adjust = "none")) # Fisher's LSD
) |>
  bind_rows(.id = "method")

# Plot p-values
methods_comparison |>
  as.data.frame() |>
  ggplot(aes(x = contrast, y = p.value, color = method, group = method)) +
  geom_point(size = 3, position = position_dodge(width = 0.5)) +
  geom_hline(yintercept = 0.05, linetype = "dashed", color = "red") +
  coord_flip() +
  labs(
    title = "P-values Across Multiple Comparison Methods",
    subtitle = "Red line shows = 0.05 threshold",
    x = "Pairwise Comparison",
    y = "P-value",
    color = "Adjustment"
  ) +
  scale_color_viridis_d(option = "plasma", end = 0.9) +
  theme_minimal() +
  theme(text = element_text(size = 12))
```

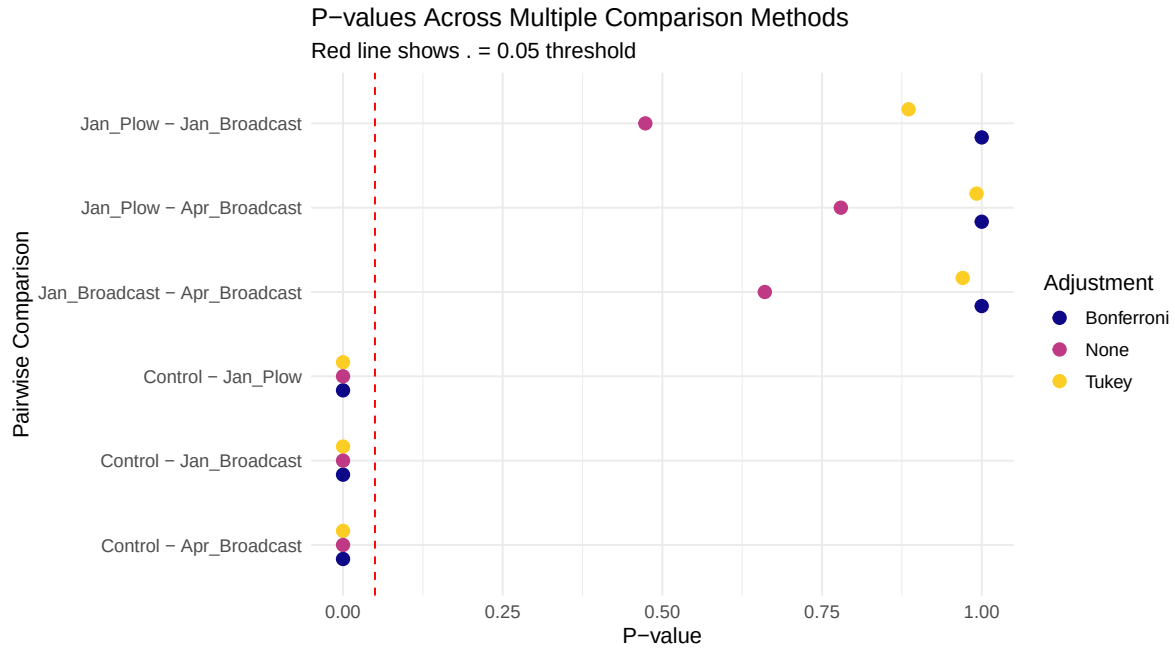


Figure 9: P-values across multiple comparison methods

Choosing the Right Method: Decision Guide

Planned Contrasts

- Pre-specified hypotheses
- Most statistical power
- Clear biological questions
- Orthogonal when possible

Example: Control vs. treatments, dose-response trends

Tukey HSD

- All pairwise comparisons
- Balanced designs
- Conservative protection

Dunnett's Test

- Compare all to control
- More power than Tukey for this case
- Common in applied trials



Tip

Best practice: Plan contrasts during experimental design. Use post-hoc only for exploratory follow-up.

Alternative: easystats for Comprehensive ANOVA Reporting

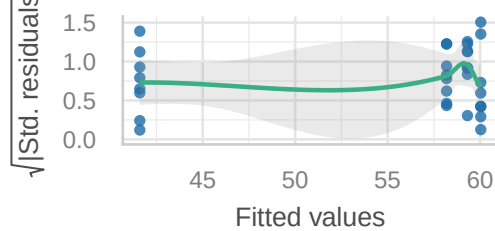
Modern integrated approach using the easystats ecosystem:

```
# Install easystats if needed: install.packages("easystats")
library(performance)
library(parameters)
library(effectsize)
library(report)

# Check model assumptions with performance::check_model()
check_model(model_npk, check = c("normality", "homogeneity", "qq", "outliers"))
```

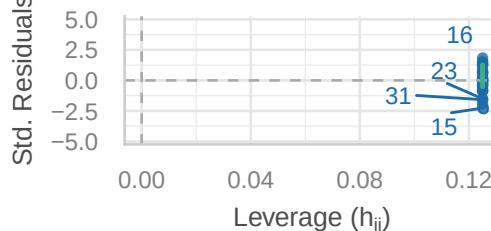
Homogeneity of Variance

Reference line should be flat and horizontal



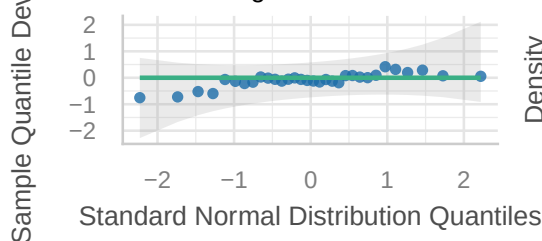
Influential Observations

Points should be inside the contour lines



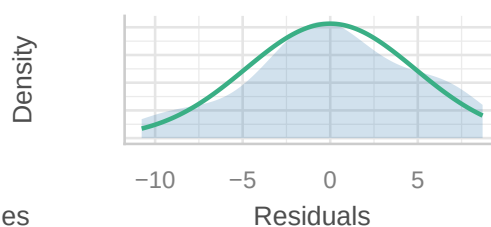
Normality of Residuals

Points should fall along the line



Normality of Residuals

Distribution should be close to the normal ci



easystats: Model Parameters & Effect Sizes

```
# Extract model parameters with confidence intervals and standardization
model_parameters(model_npk, effects = "fixed", ci = 0.95)
```

#> Parameter	Coefficient	SE	95% CI	t(28)	p
(Intercept)	41.60	1.80	[37.92, 45.28]	23.16	< .001
treatment [Jan_Plow]	18.44	2.54	[13.23, 23.64]	7.26	< .001
treatment [Jan_Broadcast]	16.59	2.54	[11.38, 21.79]	6.53	< .001
treatment [Apr_Broadcast]	17.72	2.54	[12.51, 22.92]	6.97	< .001

```
# Calculate effect sizes - proportion of variance explained by treatment
# 2 (eta-squared): Proportion of total variance explained (biased, overestimates)
# 2 (omega-squared): Less biased estimate, adjusts for sample size
# 2 (epsilon-squared): Alternative unbiased estimate
# All range from 0 (no effect) to 1 (treatment explains all variance)
eta_squared(model_npk, partial = TRUE, ci = 0.95)
```

```
#> # Effect Size for ANOVA
#>
#> Parameter | Eta2 |      95% CI
#> -----
#> treatment | 0.72 | [0.55, 1.00]
#>
#> - One-sided CIs: upper bound fixed at [1.00].
```

```
omega_squared(model_npk, partial = TRUE, ci = 0.95)
```

```
#> # Effect Size for ANOVA
#>
#> Parameter | Omega2 |      95% CI
#> -----
#> treatment | 0.68 | [0.49, 1.00]
#>
#> - One-sided CIs: upper bound fixed at [1.00].
```

Effect Size Interpretation:

- ² 0.68, ² 0.68 (treatment explains ~68% of variance)

Recommendation: Report ω^2 (omega-squared) as it's less biased than η^2 for making inferences about population effects.

easystats: Automated Reporting

Generate publication-ready text automatically:

```
# Comprehensive automated report
report(model_npk)
```

```
#> We fitted a linear model (estimated using OLS) to predict yield with treatment
#> (formula: yield ~ treatment). The model explains a statistically significant
#> and substantial proportion of variance (R2 = 0.72, F(3, 28) = 24.12, p < .001,
#> adj. R2 = 0.69). The model's intercept, corresponding to treatment = Control,
#> is at 41.60 (95% CI [37.92, 45.28], t(28) = 23.16, p < .001). Within this
#> model:
#>
#> - The effect of treatment [Jan_Flow] is statistically significant and positive
#> (beta = 18.44, 95% CI [13.23, 23.64], t(28) = 7.26, p < .001; Std. beta = 2.02,
#> 95% CI [1.45, 2.59])
#> - The effect of treatment [Jan_Broadcast] is statistically significant and
#> positive (beta = 16.59, 95% CI [11.38, 21.79], t(28) = 6.53, p < .001; Std.
#> beta = 1.81, 95% CI [1.25, 2.38])
#> - The effect of treatment [Apr_Broadcast] is statistically significant and
#> positive (beta = 17.72, 95% CI [12.51, 22.92], t(28) = 6.97, p < .001; Std.
#> beta = 1.94, 95% CI [1.37, 2.51])
#>
#> Standardized parameters were obtained by fitting the model on a standardized
#> version of the dataset. 95% Confidence Intervals (CIs) and p-values were
#> computed using a Wald t-distribution approximation.
```

easystats advantages:

- **Integrated workflow:** All aspects covered (assumptions → parameters → effect sizes → reporting)
- **Automated text:** Generate publication-ready sentences
- **Consistent syntax:** Same functions work across different model types
- **Lightweight:** Minimal dependencies compared to similar packages

Summary & Key Takeaways

1. **Check assumptions first:** Diagnostic plots are essential
2. **Planned contrasts** answer specific questions with maximum power
3. **Orthogonal contrasts** partition treatment variation into independent components
4. **emmeans** provides flexible framework for means and contrasts
5. **Post-hoc tests** control familywise error when exploring all differences
6. **Method choice matters:** Tukey (conservative), Dunnett (vs. control only)

Note

For your experiments: Design contrasts during the planning phase, not after seeing data!

References & Further Reading

Required Reading:

- Oehlert (2010), *A First Course in Design and Analysis of Experiments*
- Scheffé (1959), *The Analysis of Variance*. Wiley.

R Documentation:

- **emmeans** package: Lenth, R. V. (2024). **emmeans**: Estimated Marginal Means. <https://CRAN.R-project.org/package=emmeans>
- **multcomp** package: Hothorn et al. (2008). Simultaneous inference in general parametric models.
- **rstatix** package: Kassambara, A. (2023). **rstatix**: Pipe-Friendly Framework for Basic Statistical Tests. <https://rpkgs.datanovia.com/rstatix/>
- **ggpubr** package: Kassambara, A. (2023). **ggpubr**: ‘ggplot2’ Based Publication Ready Plots. <https://rpkgs.datanovia.com/ggpubr/>
- **easystats** ecosystem: Lüdtke et al. (2022). **easystats**: Framework for Easy Statistical Modeling, Visualization, and Reporting. <https://easystats.github.io/easystats/>

Additional Resources:

- **agridat** package vignettes with agricultural examples: <https://kwstat.github.io/agri-dat/>
- **R Script:** All code from this lecture available in `lecture05_week03_script.R`